

Manipulation de très grands nombres

3h - Sur machine

Année 2025-2026

Préambule

- Pour tester dans utop : **open Be;;** puis ouverture des modules associés aux fichiers dans lesquels il y a les fonctions que vous voulez tester.
- Le code rendu doit **impérativement compiler**. Si une partie ne compile pas, mettez-la en commentaires, cela peut donner lieu à des points.
- Interdiction de modifier les signatures (nom des fonctions + type) indiquées dans les contrats.
- Même si la non utilisation d'itérateur sera pénalisée, des points seront accordés si les fonctions sont écrites correctement sans itérateur.
- Les documents autorisés sont : vos notes de cours, TD et TP, le site d'OCaml et Moodle. Vous êtes autorisé (et incité !) à utiliser toutes les fonctions de la librairie standard, en particulier le module List : <https://ocaml.org/api/List.html>.
- L'utilisation de modèles de langage (ChatGPT, Copilot ...) et d'outils de partage de code est interdite.
- Utilisez le script `genererRendu.sh` pour générer un fichier `<votre_login>.tar` compression d'un répertoire `<votre_login>` contenant les fichiers `.ml` et les fichiers `dune`. Avant de déposer le tar, vérifiez qu'il contient bien les fichiers souhaités.

Barème prévisionnel

Exercice	1	2	3	4	5	6	7	8	9	10	11	12
Points	2.5	3	2.5	1.5	1.5	1.5	1.5	1.5	2	1.5	1.5	1.5
Total : 22 points.												

1 Manipuler des grand nombres

En informatique, on manipule généralement des nombres qui sont codés sur une quantité finie et fixe de bits. Cela signifie très concrètement que les nombres que l'on manipule ont une limite basse et haute (en l'occurrence, -2^{n-1} et $2^{n-1}-1$, respectivement, avec n le nombre de bits), au-delà desquelles le comportement n'est plus défini.

Pourtant, on a parfois besoin de manipuler des entiers très grands, beaucoup plus grands que ce que permettent les entiers codés sur un nombre fixe de bits. Pensons par exemple au nombre de Graham, nombre colossal utilisé dans la démonstration du théorème éponyme, ou au *high score* de certains joueurs de Balatro, tout aussi important.

La question se pose alors : comment peut-on manipuler de très grands nombres entiers ? Quel est le surcoût engendré ?

2 Mise en jambe : des entiers pas si grand

L'interface `IGrandNombre` du fichier `GrandNombre.ml` propose la spécification d'un grand nombre. On y trouve :

- Le type support du grand nombre

- Des fonctions de conversion, d'un entier normal vers un grand nombre (`from_int`) et d'une suite de chiffres vers un grand nombre (`from_digits`)
- Une fonction d'affichage (`afficher`)
- Une fonction de comparaison (`comparer`, attention au contrat)
- Les opérateurs binaires classiques sur les nombres : addition, soustraction, multiplication et puissance

Penchons-nous sur une première implantation « très très naïve » de très grands nombres, qui utilise... des nombres entiers classiques. Cela nous permettra de constater les limites de ce système.

- ▷ **Exercice 1** Dans le fichier `PasSiGrandNombre.ml`, proposez une réalisation de l'interface `IGrandNombre` dont le type support est le type `t = int`. À noter que plusieurs fonctions ont déjà été implantées. (Attention, il vous faudra implémenter la puissance vous-même!).

Avec ces quelques opérations, on peut déjà écrire des petits algorithmes !

- ▷ **Exercice 2** Dans le module foncteur `GrandNombreAlgorithmes` (fichier `GrandNombre.ml`), implémenter les algorithmes de la factorielle et du PGCD en utilisant les opérations disponibles dans `I`, paramètre du foncteur. On veillera à bien respecter le **contrat** de ces fonctions.

Tests associés aux exercices 1 et 2 Vous remarquerez des appels aux foncteurs `GrandNombreAlgorithmes` et `GrandNombreTest` en bas du fichier `PasSiGrandNombre.ml`, ce qui permet de lancer les tests unitaires lorsque l'on fait `dune runtest`. Pensez à les décommenter !

Lancer les tests se charge également d'afficher quelques calculs sur la console (en plusieurs fois, c'est normal). On constate alors que l'utilisation des nombres entiers est limitée dès qu'on a des opérations aux résultats très grands, comme $25!$ ou 1446^{131} ... Voyons maintenant comment pallier cela.

3 Liste d'entiers

Nous allons nous pencher sur une solution assez simple pour réaliser des grands nombres : les listes d'entiers.

Concrètement, nous allons stocker les chiffres composant un nombre dans une liste, en suivant une base quelconque. Plus la base est grande et plus l'implantation est efficace (car la liste est moins grande pour un même nombre). Pour simplifier la suite, nous allons prendre la **base 100**, et stocker les *chiffres de poids faible en premier*.

$$\overline{a_n a_{n-1} \dots a_1 a_0}^b = a_n b^n + a_{n-1} b^{n-1} + \dots + a_1 b + a_0 = [a_0; a_1; \dots; a_{n-1}; a_n]$$

$$1234567 = 1 \times 100^3 + 23 \times 100^2 + 45 \times 100 + 67 = [67; 45; 23; 1]$$

$$80907 = 8 \times 100^2 + 9 \times 100 + 7 = [7; 9; 8]$$

FIGURE 1 – Représentation d'entiers sous forme de listes

En plus de cette liste d'entiers, on a besoin de stocker le *signe* du nombre considéré. Pour cela, on utilisera un booléen, qui vaudra `true` si le nombre est négatif et `false` sinon. Le type final pour représenter nos grands nombres est donc `bool * int list`.

À noter que, canoniquement, le nombre 0 est représenté par la liste vide (`[]`), avec le signe plus ou moins, peu importe.

Les exercices de cette section sont à réaliser dans le fichier `IntList.ml`

Note : nous vous fournissons les implémentations d'une fonction qui supprime les 0 surnuméraires (`normalise`) ainsi que la fonction d'affichage (`afficher`).

3.1 Conversion

Penchons-nous d'abord sur comment implémenter les fonctions de base du module.

- ▷ **Exercice 3** Écrivez les tests et la fonction `from_int` qui transforme un entier quelconque en grand nombre (booléen + liste d'entiers entre 0 et 99).
- ▷ **Exercice 4** Écrivez la fonction `from_digits` qui transforme une suite de chiffres (entre 0 et 9) en grand nombre. Vous ne devez pas utiliser la fonction `from_int`.

3.2 Comparaison

Abordons la comparaison de grands nombres. Comme souvent avec ce genre de types composites, nous allons d'abord écrire des fonctions *sur les listes de nombres*, puis nous écrirons les fonctions sur les grands nombres, qui exploiteront les fonctions écrites auparavant.

- ▷ **Exercice 5** Écrivez la fonction `comparer_listes` qui compare deux listes de nombres. On pourra pour cela exploiter le fait que :
 - si les listes sont de tailles différentes, alors on peut les comparer sans regarder leur contenu ;
 - sinon, on peut utiliser l'ordre lexicographique sur les listes en partant de la droite : on compare les éléments tout au bout de la liste, si ils sont identiques on compare les éléments juste avant, et ainsi de suite.(et se convaincre que la fonction de comparaison obtenue est la même que l'ordre habituel sur les entiers!).
- ▷ **Exercice 6** Écrivez la fonction `comparer` qui compare deux grands nombres quelconques.

3.3 Addition et soustraction

Penchons-nous maintenant sur les opérations d'addition et de soustraction. Exactement comme pour la comparaison, nous allons d'abord écrire des fonctions sur les *listes*, puis des fonctions sur les grands nombres, exploitant ces fonctions.

- ▷ **Exercice 7** Écrivez la fonction `plus_listes` qui réalise la somme de deux listes de nombres en suivant le principe des additions « posées » : on somme les chiffres les plus à droite, on obtient le chiffre le plus à droite du résultat et une retenue (éventuellement 0), on somme les chiffres qui suivent et la retenue, on obtient le chiffre suivant et une nouvelle retenue, et ainsi de suite.
- ▷ **Exercice 8** Écrivez la fonction `moins_listes` qui réalise la différence entre deux listes de nombres **avec le premier nombre plus grand que le second**, en suivant le principe de la soustraction posée (similaire à l'addition posée).
- ▷ **Exercice 9** Implémentez maintenant les fonctions `plus` et `moins`, qui prennent en entrée des grands nombres quelconque et retourne leur somme et leur différence, respectivement.

3.4 Multiplication

La multiplication suit un peu la même logique que l'addition et la soustraction : nous allons effectivement *poser* la multiplication. Dans un premier temps, nous devons multiplier le premier facteur par chaque « chiffre » du second, puis additionner chaque nombre ainsi obtenu en les décalant sur la gauche.

				7	89	
×		1	23	45		
		3	55	05	(789 × 45)	
	1	81	47		(789 × 23)	
	7	89			(789 × 1)	
	9	74	02	05		

Pour réaliser chaque multiplication partielle on peut procéder similairement à l'addition déjà vue : on multiplie le chiffre le plus à droite par le chiffre seul, on obtient le chiffre le plus à droit du résultat et une retenue, puis on multiplie le chiffre suivant par le chiffre seul, au résultat duquel on ajoute la retenue, on obtient le chiffre suivant du résultat et une retenue, etc.

- ▷ **Exercice 10** Écrivez la fonction `mult_coeff` qui réalise la multiplication d'une liste de chiffres par un chiffre seul.
- ▷ **Exercice 11** Écrivez la fonction `mult`, qui réalise la multiplication de deux grands nombres quelconques.

3.5 Puissance

- ▷ **Exercice 12** Écrivez la fonction `puiss` qui réalise la puissance de deux grands nombres, avec l'exposant nécessairement positif ou nul.